

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ án Đa ngành hướng Trí tuệ Nhân tạo

Báo cáo đề tài

Hệ Thống Cảnh Báo Vật Cản Bằng Giọng Nói Cho Người Khiếm Thị Ứng Dụng Mạng TinyML

Giảng viên hướng dẫn: Tiến sĩ Võ Tuấn Bình

Sinh viên thực hiện:	Nguyễn Đăng Hiên	2310936
	Lê Nguyễn Kim Khôi	2311671
	Bùi Ngọc Phúc	2312665
	Hồ Anh Dũng	2310543
	Cao Hữu Thiên Hoàng	2311030

THÀNH PHỐ HỒ CHÍ MINH, THÁNG 5 NĂM 2026

Tóm tắt

Đề tài xây dựng hệ thống cảnh báo vật cản bằng giọng nói cho người khiếm thị, sử dụng camera để thu nhận hình ảnh phía trước, mô hình EfficientDet-Lite0 để nhận diện vật thể và Decision Engine để chọn cảnh báo quan trọng nhất. Hệ thống tập trung vào pipeline tích hợp trên thiết bị biên: thu nhận ảnh, suy luận object detection, lọc vật thể liên quan đến an toàn di chuyển, ước lượng mức rủi ro va chạm ngắn hạn, xác định hướng trái–giữa–phải và phát thông báo bằng Text-to-Speech.

Trong phạm vi hiện tại, ESP32-CAM được dùng như nguồn camera qua mạng, còn mô hình AI chạy trên Raspberry Pi hoặc máy tính thử nghiệm. Vì vậy, thuật ngữ TinyML trong báo cáo được hiểu theo hướng sử dụng mô hình nhẹ và suy luận ở biên, không khẳng định mô hình đã chạy trực tiếp trên vi điều khiển ESP32-CAM. Về phương pháp luận, hệ thống được định hướng theo khái niệm time-to-contact (TTC) và temporal geofence của Badki et al.^[1]: thay vì cố đo khoảng cách tuyệt đối theo mét, hệ thống ưu tiên ước lượng xem một vật thể có đang tiến gần đến vùng va chạm trong ngắn hạn hay không. Báo cáo trình bày kiến trúc, thuật toán ra quyết định, cấu trúc phần mềm, kịch bản kiểm thử và các chỉ số cần đo thực nghiệm trước khi đánh giá khả năng sử dụng thực tế.



Mục lục

Tóm tắt	3
1 Mở đầu	6
1.1 Bối cảnh và lý do chọn đề tài	6
1.2 Mục tiêu	6
1.3 Phạm vi và giới hạn	6
2 Cơ sở công nghệ	7
2.1 TinyML và xử lý biên	7
2.2 EfficientDet-Lite0 và TensorFlow Lite	7
2.3 MediaPipe Object Detector	7
2.4 Time-to-Contact và temporal geofence	7
2.5 Text-to-Speech và giao diện giám sát	8
3 Phân tích yêu cầu và kịch bản sử dụng	8
3.1 Người dùng mục tiêu	8
3.2 Yêu cầu chức năng	9
3.3 Kịch bản tiêu biểu	9
4 Thiết kế hệ thống	9
4.1 Kiến trúc tổng thể	9
4.2 Luồng xử lý dữ liệu	10
5 Thiết kế Decision Engine	10
5.1 Vai trò	10
5.2 Lọc lớp vật thể	10
5.3 Từ khoảng cách tương đối sang TTC proxy	11
5.4 Xác định hướng trái–giữa–phải	12
5.5 Tính điểm ưu tiên	12
5.6 Sinh câu cảnh báo và chống lặp âm thanh	12
6 Triển khai phần mềm	13
6.1 Cấu trúc module	13
6.2 Cấu hình quan trọng	13
6.3 Hỗ trợ ESP32-CAM	13



7	Kiểm thử và đánh giá	14
7.1	Mục tiêu kiểm thử	14
7.2	Chỉ số cần đo thực nghiệm	15
8	Đánh giá hạn chế và rủi ro	15
8.1	Hạn chế hiện tại	15
8.2	Rủi ro khi triển khai thực tế	15
9	Hướng phát triển	16
10	Kết luận	16

Danh sách hình

4.1	Kiến trúc xử lý từ camera đến cảnh báo giọng nói	10
-----	--	----

Danh sách bảng

3.1	Yêu cầu chức năng chính	9
5.1	Nhóm vật thể ưu tiên cảnh báo	11
6.1	Vai trò các module chính	13
7.1	Kịch bản kiểm thử đề xuất	14
7.2	Bảng chỉ số thực nghiệm cần cập nhật sau khi chạy demo	15

1 Mở đầu

1.1 Bối cảnh và lý do chọn đề tài

Người khiếm thị thường cần thêm thông tin về các vật cản phía trước khi di chuyển trong hành lang, sân trường, vỉa hè hoặc khu vực công cộng. Các công cụ hỗ trợ truyền thống như gậy dò đường cho phản hồi trực tiếp nhưng phạm vi phát hiện bị giới hạn ở vùng tiếp xúc gần. Trong khi đó, camera kết hợp thị giác máy tính có thể cung cấp thông tin sớm hơn về người đi bộ, xe máy, xe đạp, ô tô hoặc các vật cản tĩnh nằm trong hướng di chuyển.

Đề tài xây dựng một hệ thống trợ thị gọn nhẹ, sử dụng camera để thu nhận hình ảnh, mô hình nhận diện vật thể cỡ nhỏ để phát hiện vật cản, lớp ra quyết định để chọn cảnh báo quan trọng nhất, và Text-to-Speech để chuyển kết quả thành giọng nói. Điểm chính của đề tài không nằm ở huấn luyện mô hình từ đầu, mà ở việc tích hợp một pipeline hoàn chỉnh, có khả năng chạy trên thiết bị nhúng hoặc máy tính biên.

1.2 Mục tiêu

Mục tiêu tổng quát là xây dựng hệ thống có khả năng phát hiện vật cản cơ bản phía trước người dùng và phát cảnh báo bằng giọng nói theo thời gian thực. Các mục tiêu kỹ thuật gồm:

- Thu nhận khung hình từ camera cục bộ hoặc ESP32-CAM.
- Nhận diện các lớp vật thể liên quan đến an toàn di chuyển như người, xe đạp, xe máy, ô tô, xe buýt, xe tải, ghế, băng ghế và vật cản tĩnh lớn.
- Ước lượng mức độ sắp-và-chạm theo TTC proxy từ chuỗi bounding box ngắn hạn, không đo khoảng cách tuyệt đối theo mét.
- Chọn một vật thể có mức nguy hiểm cao nhất để tránh phát quá nhiều cảnh báo.
- Phát cảnh báo giọng nói ngắn gọn, có cơ chế cooldown để hạn chế lặp âm thanh.

1.3 Phạm vi và giới hạn

Trong phạm vi đồ án, nhóm sử dụng mô hình pre-trained EfficientDet-Lite0 thay vì tự thu thập dữ liệu và huấn luyện từ đầu. Hệ thống không hồi quy TTC bằng một mạng temporal chuyên dụng như trong^[1]; thay vào đó, nhóm dùng TTC proxy từ sự tăng kích thước biểu kiến của bounding box qua các khung hình liên tiếp. ESP32-CAM chưa phải

thiết bị chạy suy luận AI, mà là nguồn cung cấp hình ảnh cho Raspberry Pi hoặc máy tính biên. Các điều kiện khó như trời tối, mưa lớn, ngược sáng mạnh hoặc môi trường quá đông người chưa được xử lý đầy đủ. Sản phẩm đóng vai trò hỗ trợ cảnh báo, không thay thế hoàn toàn các công cụ di chuyển an toàn khác.

2 Cơ sở công nghệ

2.1 TinyML và xử lý biên

TinyML là hướng triển khai mô hình học máy có kích thước nhỏ trên thiết bị tài nguyên hạn chế. Với bài toán cảnh báo vật cản, xử lý trực tiếp gần camera giúp giảm phụ thuộc vào máy chủ từ xa và giảm độ trễ truyền dữ liệu. Trong đề tài này, hướng TinyML được hiểu là dùng mô hình nhẹ EfficientDet-Lite0 và suy luận ở thiết bị biên như Raspberry Pi hoặc máy tính thử nghiệm; ESP32-CAM chỉ đóng vai trò nguồn camera qua mạng, chưa chạy trực tiếp mô hình AI.

2.2 EfficientDet-Lite0 và TensorFlow Lite

EfficientDet là họ mô hình object detection được thiết kế để cân bằng độ chính xác và chi phí tính toán^[5]. Phiên bản EfficientDet-Lite0 có kích thước nhỏ hơn, phù hợp với các thiết bị biên và được cung cấp sẵn ở định dạng TensorFlow Lite/MediaPipe^[6]. Trong hệ thống này, mô hình trả về danh sách detection gồm nhãn, độ tin cậy và bounding box.

2.3 MediaPipe Object Detector

MediaPipe cung cấp API Object Detector cho Python, giúp nạp mô hình `.tflite`, nhận ảnh đầu vào và trả về kết quả nhận diện ở dạng có cấu trúc^[2]. Nhờ đó, module `detector.py` chỉ cần tập trung vào chuyển đổi frame BGR sang RGB, chạy suy luận và chuẩn hóa kết quả thành các trường `label`, `score`, `x`, `y`, `w`, `h`.

2.4 Time-to-Contact và temporal geofence

Time-to-contact (TTC) là thời gian còn lại trước khi một vật thể chạm đến mặt phẳng camera dưới điều kiện chuyển động hiện tại. Badki et al. cho thấy TTC là một tín hiệu hữu ích cho điều hướng vì nó gần với quyết định “có cần phản ứng ngay hay không” hơn là chỉ đo depth tức thời^[1]. Paper này diễn đạt bài toán dưới dạng temporal geofence: hệ

thống dự đoán liệu vật thể có đi vào vùng va chạm trong một ngưỡng thời gian nhất định hay không, thay vì yêu cầu khoảng cách tuyệt đối.

Trong đề án này, nhóm không tái hiện đầy đủ mạng học temporal của paper. Thay vào đó, tư tưởng TTC được adapt ở mức engineering: theo dõi cùng một vật thể trong vài khung hình liên tiếp, quan sát bounding box có mở rộng hay không, và dùng tốc độ mở rộng đó như một TTC proxy để nâng mức cảnh báo.

2.5 Text-to-Speech và giao diện giám sát

Thư viện `pyttsx3` được dùng để phát giọng nói cục bộ, không cần gọi dịch vụ đám mây. Ngoài giao diện OpenCV^[3], đề tài còn có Flask UI^[4] để quan sát live stream, bật/tắt detection và kiểm tra trạng thái hệ thống qua trình duyệt.

3 Phân tích yêu cầu và kịch bản sử dụng

3.1 Người dùng mục tiêu

Người dùng mục tiêu là người khiếm thị hoặc người có thị lực rất kém, cần di chuyển trong các không gian quen thuộc như hành lang trường học, khuôn viên trường, vỉa hè, lối đi bộ hoặc khu dân cư. Thiết bị được giả định gắn trước ngực hoặc cầm theo hướng camera nhìn về phía trước.

3.2 Yêu cầu chức năng

Bảng 3.1: Yêu cầu chức năng chính

Mã	Yêu cầu
F1	Hệ thống đọc frame từ camera cục bộ hoặc ESP32-CAM.
F2	Hệ thống chạy object detection theo chu kỳ, không nhất thiết suy luận mọi frame.
F3	Hệ thống lọc các lớp vật thể không liên quan đến an toàn di chuyển.
F4	Hệ thống ước lượng vùng trái–giữa–phải và mức rủi ro TTC gần hạn của vật thể.
F5	Hệ thống chọn vật thể có độ ưu tiên cao nhất để phát cảnh báo.
F6	Hệ thống phát giọng nói cục bộ và hạn chế lặp cảnh báo liên tục.
F7	Hệ thống có giao diện quan sát luồng camera và trạng thái detection.

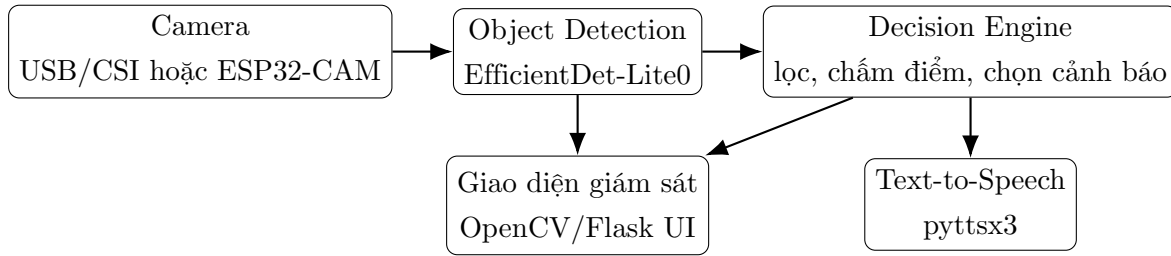
3.3 Kịch bản tiêu biểu

- Hành lang hoặc sân trường: hệ thống phát hiện người hoặc ghế chắn giữa lối đi và cảnh báo để người dùng đổi hướng.
- Vĩa hè hoặc lối đi ngoài trời: hệ thống phát hiện xe máy, xe đạp hoặc vật cản lớn nằm ở mép đường.
- Vật cản xuất hiện gần: khi người hoặc phương tiện xuất hiện gần và nằm trong vùng trung tâm, hệ thống ưu tiên phát cảnh báo sớm.
- Nhiều vật thể cùng lúc: hệ thống không đọc toàn bộ nhãn, mà chọn đối tượng có điểm nguy hiểm cao nhất.

4 Thiết kế hệ thống

4.1 Kiến trúc tổng thể

Hệ thống gồm bốn khối chính: đầu vào hình ảnh, nhận diện vật thể, ra quyết định cảnh báo và đầu ra giọng nói/giao diện. Kiến trúc này biến kết quả object detection thô thành thông tin cảnh báo có ý nghĩa sử dụng thực tế.



Hình 4.1: Kiến trúc xử lý từ camera đến cảnh báo giọng nói

4.2 Luồng xử lý dữ liệu

Luồng xử lý chính bắt đầu từ frame camera, sau đó resize về kích thước cấu hình, chạy suy luận theo chu kỳ `INFER_EVERY_N_FRAMES`, trích xuất danh sách detection và đưa vào Decision Engine. Nếu có vật thể đủ điều kiện, hệ thống sinh câu cảnh báo và chuyển cho Voice Alert Manager. OpenCV hoặc Flask UI hiển thị frame, FPS và bounding box của vật thể được chọn.

Camera → tiền xử lý frame → EfficientDet-Lite0 → danh sách detection → ghép object ngắn hạn qua các frame → ước lượng TTC proxy và risk score → chọn vật thể ưu tiên → sinh câu cảnh báo → phát giọng nói

5 Thiết kế Decision Engine

5.1 Vai trò

Nếu đọc trực tiếp mọi nhãn mà mô hình phát hiện được, hệ thống dễ tạo ra quá nhiều thông báo và gây nhiễu cho người dùng. Decision Engine giải quyết vấn đề này bằng cách trả lời bốn câu hỏi: vật thể nào liên quan đến an toàn di chuyển, vật thể đó ở gần hay xa, nó nằm ở hướng nào trong khung hình, và vật thể nào cần được cảnh báo trước.

5.2 Loại lớp vật thể

Hệ thống chỉ giữ lại các lớp có trọng số trong `CLASS_WEIGHTS`. Các lớp này được chia thành nhóm ưu tiên theo mức ảnh hưởng đến di chuyển.



Bảng 5.1: Nhóm vật thể ưu tiên cảnh báo

Nhóm	Ví dụ lớp vật thể
Ưu tiên cao	person, bicycle, motorcycle, car, bus, truck
Ưu tiên trung bình	chair, bench, suitcase, potted plant, backpack
Bỏ qua giai đoạn đầu	bottle, cup, remote, tv và các vật nhỏ ít ảnh hưởng đến lối đi

5.3 Từ khoảng cách tương đối sang TTC proxy

Do hệ thống dùng một camera RGB monocular, khoảng cách tuyệt đối không được đo trực tiếp. Nếu chỉ dùng diện tích bounding box của một ảnh đơn, hệ thống dễ nhầm giữa vật thể “trông lớn” và vật thể “đang tiến nhanh vào vùng va chạm”. Vì vậy, nhóm chuyển bài toán ra quyết định từ distance heuristic sang TTC proxy.

Ý tưởng là theo dõi cùng một object qua các khung hình liên tiếp bằng phép ghép IoU đơn giản theo cùng nhãn lớp. Với mỗi object đã ghép thành công, hệ thống tính một đại lượng kích thước biểu kiến:

$$s_t = 0.7 \cdot \frac{h_t}{H} + 0.3 \cdot \sqrt{\frac{w_t h_t}{WH}}$$

trong đó w_t, h_t là kích thước bounding box tại thời điểm t , còn W, H là kích thước frame. Thành phần chiều cao được đặt trọng số lớn hơn vì ổn định hơn diện tích thuần khi object bị crop hoặc thay đổi tỉ lệ khung.

Nếu $s_t > s_{t-1}$, vật thể đang mở rộng trong ảnh và có xu hướng tiến lại gần camera. Từ đó, hệ thống ước lượng TTC proxy:

$$\widehat{TTC} \approx \frac{\Delta t}{(s_t - s_{t-1})/s_{t-1}}$$

với Δt là thời gian giữa hai lần suy luận liên tiếp. TTC proxy nhỏ nghĩa là tốc độ mở rộng cao và nguy cơ va chạm ngắn hạn lớn hơn. Để giảm dao động, hệ thống làm mượt TTC proxy bằng exponential smoothing trước khi đưa vào quyết định cuối.

Trong những frame đầu hoặc khi object chưa ghép được track ổn định, hệ thống dùng spatial fallback dựa trên diện tích bounding box và vị trí đáy box. Điều này giúp hệ thống vẫn phản ứng được ngay cả khi chưa đủ dữ liệu temporal.

5.4 Xác định hướng trái–giữa–phải

Tâm bounding box theo trục ngang được chuẩn hóa theo chiều rộng frame. Nếu tâm nằm dưới 0.33, vật thể được xem là ở bên trái; nếu trên 0.67, vật thể ở bên phải; phần còn lại là vùng giữa. Vùng giữa có trọng số cao hơn vì thường trùng với hướng di chuyển chính.

5.5 Tính điểm ưu tiên

Mỗi detection hợp lệ được chuyển thành một `ScoredObstacle`. Nếu TTC proxy khả dụng, hệ thống ánh xạ nó về ba mức rủi ro gần hạn: *near*, *medium*, *far* theo các ngưỡng thời gian. Nếu TTC chưa khả dụng, hệ thống quay về spatial fallback. Sau đó, điểm ưu tiên được tính bằng tích của trọng số lớp, trọng số TTC/spatial risk, trọng số vùng trung tâm, trọng số hướng và độ tin cậy:

$$\begin{aligned} Priority = & ClassWeight \times RiskWeight \times CenterWeight \\ & \times DirectionWeight \times Confidence \end{aligned}$$

Vật thể có *Priority* cao nhất được chọn làm cảnh báo chính tại thời điểm hiện tại. Cách chấm điểm dạng nhân giúp vật thể nguy hiểm thật sự, ví dụ người hoặc xe đang tiến nhanh trong vùng trung tâm, nổi bật hơn vật thể đứng yên ở rìa khung hình.

5.6 Sinh câu cảnh báo và chống lặp âm thanh

Decision Engine sinh câu cảnh báo ngắn gọn theo mức rủi ro. Khi TTC proxy cho thấy vật thể đang tiến nhanh, câu cảnh báo chuyển sang dạng mạnh hơn như “Warning, motorcycle approaching fast on the right”. Module `VoiceAlertManager` chỉ phát lại khi qua thời gian cooldown hoặc khi mức nguy hiểm tăng lên. Cơ chế này giúp người dùng nhận thông tin cần thiết mà không bị lặp âm thanh liên tục.



6 Triển khai phần mềm

6.1 Cấu trúc module

Bảng 6.1: Vai trò các module chính

Module	Vai trò
<code>main.py</code>	Điều phối camera, detector, decision engine, TTS và giao diện OpenCV.
<code>detector.py</code>	Nạp EfficientDet-Lite0 bằng MediaPipe Object Detector và chuẩn hóa detection.
<code>decision_engine.py</code>	Lọc vật thể, ghép track ngắn hạn, ước lượng TTC proxy, chia vùng trái-giữa-phải, tính priority và sinh câu cảnh báo.
<code>tts_engine.py</code>	Quản lý Text-to-Speech bằng <code>pyttsx3</code> , cooldown và phát lại khi rủi ro tăng.
<code>esp32_camera.py</code>	Hỗ trợ đọc camera qua MJPEG stream hoặc HTTP JPG polling.
<code>web_ui.py</code>	Cung cấp Flask UI để quan sát live stream và bật/tắt detection.
<code>config.py</code>	Chứa cấu hình model, camera, ngưỡng score, trọng số lớp, ngưỡng gần-xa và nhân tiếng Việt.

6.2 Cấu hình quan trọng

Các tham số quan trọng được gom vào `config.py`, gồm kích thước frame, chu kỳ suy luận, ngưỡng confidence, số detection tối đa, các ngưỡng spatial fallback, ngưỡng IoU để ghép track ngắn hạn, ngưỡng TTC proxy và thời gian cooldown giọng nói. Việc đưa các giá trị này vào file cấu hình giúp nhóm dễ tinh chỉnh khi chuyển từ máy tính thử nghiệm sang Raspberry Pi hoặc khi đổi camera.

6.3 Hỗ trợ ESP32-CAM

Module ESP32 camera tự thử hai chế độ đọc ảnh: MJPEG stream bằng OpenCV và HTTP JPG polling bằng `requests`. Cách làm này tăng khả năng tương thích với nhiều

firmware ESP32-CAM, vì mỗi firmware có thể cung cấp endpoint khác nhau như `/stream`, `/jpg` hoặc `/mjpeg`.

7 Kiểm thử và đánh giá

7.1 Mục tiêu kiểm thử

Kiểm thử nhằm xác nhận hệ thống không chỉ nhận diện vật thể, mà còn chọn đúng vật thể cần cảnh báo, phát đúng hướng, phản ứng đúng theo TTC proxy và không lặp âm thanh gây nhiễu.

Bảng 7.1: Kịch bản kiểm thử đề xuất

STT	Tình huống	Kết quả mong đợi	Tiêu chí đánh giá
1	Một người đứng giữa lối đi ở khoảng cách gần	Phát cảnh báo người ở giữa hoặc phía trước	Nhận diện đúng lớp person, priority cao
2	Xe máy nằm ở mép phải khung hình	Phát cảnh báo xe máy bên phải	Xác định đúng vùng phải, không ưu tiên vật nhỏ khác
3	Ghế ở xa trong hành lang	Cảnh báo mức thấp hơn hoặc bỏ qua nếu không nằm trên hướng chính	Spatial fallback và vùng trung tâm hợp lý
4	Nhiều vật thể cùng lúc: người ở giữa, xe đạp ở rìa trái	Ưu tiên người ở giữa	Thuật toán chọn vật thể ưu tiên đúng
5	Một vật thể đứng yên trong nhiều giây	Không tự tăng cảnh báo nếu bbox gần như không mở rộng	TTC proxy không báo động giả
6	Một người đi nhanh từ xa tiến vào giữa khung hình	Mức cảnh báo tăng dần từ far lên medium/near	TTC proxy phản ánh xu hướng tiến gần

7.2 Chỉ số cần đo thực nghiệm

Nhóm chưa nên ghi số liệu định lượng nếu chưa đo trực tiếp trên thiết bị chạy demo. Các chỉ số cần đo khi nghiệm thu gồm:

Bảng 7.2: Bảng chỉ số thực nghiệm cần cập nhật sau khi chạy demo

Chỉ số	Cách đo	Giá trị hiện tại
FPS trung bình	Ghi nhận FPS hiển thị trong vòng 60 giây với cùng cấu hình camera	Cần đo
Độ trễ suy luận	Thời gian từ lúc nhận frame đến khi có danh sách detection	Cần đo
Độ trễ end-to-end	Thời gian từ lúc vật cản xuất hiện đến khi phát âm thanh	Cần đo
Tỷ lệ cảnh báo đúng vật thể ưu tiên	Số tình huống chọn đúng vật nguy hiểm nhất trên tổng số tình huống test	Cần đo
Tỷ lệ lặp cảnh báo không mong muốn	Số lần phát lặp trong thời gian cooldown	Cần đo

8 Đánh giá hạn chế và rủi ro

8.1 Hạn chế hiện tại

Hệ thống chưa đo được khoảng cách tuyệt đối theo mét vì chỉ dùng một camera RGB. TTC proxy hiện tại cũng chưa phải ước lượng TTC học sâu đúng nghĩa như trong^[1], mà chỉ là xấp xỉ engineering từ tracked bounding box. Kết quả nhận diện phụ thuộc vào ánh sáng, góc nhìn, độ phân giải camera và chất lượng luồng hình ảnh từ ESP32-CAM. Nếu dùng ESP32-CAM qua Wi-Fi, độ trễ mạng có thể ảnh hưởng đến thời điểm phát cảnh báo. Ngoài ra, `pyttssx3` có thể làm chậm luồng xử lý nếu phát âm thanh đồng bộ trong cùng tiến trình.

8.2 Rủi ro khi triển khai thực tế

Các rủi ro quan trọng gồm cảnh báo sai lớp vật thể, bỏ sót vật cản thấp không có trong tập lớp COCO, lặp cảnh báo gây mất tập trung, và trễ tổng thể quá lớn khi người

dùng di chuyển nhanh. Vì vậy, hệ thống cần được xem là công cụ hỗ trợ thử nghiệm, chưa đủ điều kiện sử dụng như thiết bị an toàn độc lập.

9 Hướng phát triển

Các hướng mở rộng phù hợp gồm:

- Nâng cấp từ TTC proxy dựa trên bounding box sang mô hình TTC temporal chuyên dụng hoặc monocular depth có hiệu chỉnh.
- Tinh chỉnh mô hình với dữ liệu thực tế tại Việt Nam, đặc biệt là xe máy, vỉa hè và vật cản thấp.
- Tối ưu TTS theo hướng bất đồng bộ để không chặn luồng camera.
- Bổ sung phản hồi rung hoặc tai nghe dẫn hướng nếu cần tăng mức hỗ trợ.
- Đo và tối ưu end-to-end latency trên Raspberry Pi thay vì chỉ kiểm tra trên laptop.

10 Kết luận

Đề tài đã xác định được một hướng triển khai khả thi cho hệ thống cảnh báo vật cản bằng giọng nói cho người khiếm thị. Thay vì chỉ dừng ở nhận diện vật thể và heuristic khoảng cách một ảnh đơn, hệ thống bổ sung Decision Engine theo hướng TTC proxy để lọc vật thể liên quan, đánh giá nguy cơ va chạm ngắn hạn và chọn một cảnh báo ưu tiên. Cách tiếp cận này phù hợp với mục tiêu của đề án đa ngành vì kết hợp phần cứng camera, mô hình AI nhẹ, xử lý phần mềm, giao diện giám sát và tương tác âm thanh.

Trong giai đoạn tiếp theo, trọng tâm cần chuyển từ hoàn thiện pipeline sang đo thực nghiệm trên thiết bị thật. Các chỉ số như FPS, độ trễ suy luận, độ trễ phát âm thanh và tỷ lệ cảnh báo đúng vật thể ưu tiên sẽ quyết định mức độ khả dụng của hệ thống trong môi trường thực tế.

Tài liệu tham khảo

- [1] Abhishek Badki, Orazio Gallo, Jan Kautz, and Pradeep Sen. Binary ttc: A temporal geofence for autonomous navigation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12946–12955, 2021.



- [2] Google AI Edge. Mediapipe object detector task guide. https://ai.google.dev/edge/mediapipe/solutions/vision/object_detector, 2026. Truy cập tháng 5 năm 2026.
- [3] OpenCV. Opencv documentation. <https://docs.opencv.org/>, 2026. Truy cập tháng 5 năm 2026.
- [4] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/>, 2026. Truy cập tháng 5 năm 2026.
- [5] Mingxing Tan, Ruoming Pang, and Quoc V. Le. Efficientdet: Scalable and efficient object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10781–10790, 2020.
- [6] TensorFlow. Tensorflow lite. <https://www.tensorflow.org/lite>, 2026. Truy cập tháng 5 năm 2026.